Matrices y Alternativas Vectoriales

https://gemini.google.com/app/c84e2e58ccf5bbe1

User prompt: explicame de manera sencilla porque las matrices son una manera de operar o interpretar vectores y que otras tecnicas hay

Response: Una matriz no es más que una **máquina de transformar vectores**: toma un vector de entrada, lo procesa según ciertas reglas geométricas o numéricas, y escupe un nuevo vector transformado.

¿Por qué las matrices operan e interpretan vectores?

Imagina el espacio geométrico habitual (2D o 3D). Cualquier punto o flecha que dibujes es un **vector** (x, y).

1. Todo se reduce a dónde aterrizan los vectores base

En 2D, cualquier vector se construye combinando dos vectores unitarios fundamentales:

- $\hat{i} = (1, 0)$ (el paso base en el eje X)
- $\hat{j} = (0, 1)$ (el paso base en el eje Y)

Cualquier vector como $(3, 2)$ significa simplemente: *"avanza 3 veces $\hat{i}$ y 2 veces $\hat{j}$"*.

Si aplicas una transformación lineal (rotar, estirar, reflejar), las líneas rectas siguen rectas y el origen no se mueve. Por tanto, para saber qué le pasa a **cualquier** vector, basta saber adónde van a parar $\hat{i}$ y $\hat{j}$.

2. Las columnas de una matriz son ese destino

Si tomas una matriz de 2×2:

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix}$$

- La primera columna $\begin{pmatrix} a \\ c \end{pmatrix}$ es **adónde fue a parar** $\hat{i}$.
- La segunda columna $\begin{pmatrix} b \\ d \end{pmatrix}$ es **adónde fue a parar** $\hat{j}$.

Cuando multiplicas la matriz por el vector $\begin{pmatrix} x \\ y \end{pmatrix}$:

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = x \begin{pmatrix} a \\ c \end{pmatrix} + y \begin{pmatrix} b \\ d \end{pmatrix}$$

Estás diciendo literalmente: *"reconstruye el vector original combinando las nuevas posiciones de los ejes"*.

3. Qué significa operar con ellas

- **Geometría:** Rotaciones, proyecciones (sombras), escalados o deformaciones.
- **Sistemas de ecuaciones:** Una matriz codifica cómo interactúan varias variables al mismo tiempo.
- **Cambio de base:** Expresar el mismo vector pero desde el punto de vista de otro sistema de coordenadas.

¿Qué otras técnicas existen para representar u operar vectores?

Las matrices son el estándar del álgebra lineal clásica, pero no son la única herramienta:

1. Cuaterniones (*Quaternions*)

- **Qué son:** Una extensión de los números complejos con tres componentes imaginarias (i, j, k).
- **Para qué sirven:** Para rotaciones en 3D.
- **Ventaja frente a matrices:** Una rotación 3D con matrices usa 9 números (3×3); un cuaternión usa solo 4. Son computacionalmente más eficientes, no sufren de *gimbal lock* (bloqueo cardánico) y permiten interpolar movimientos suaves en motores gráficos y robótica (SLERP).

2. Álgebra Geométrica / Álgebra de Clifford

- **Qué es:** Un sistema que unifica escalares, vectores, áreas orientadas (*bivectores*) y volúmenes orientados (*trivectores*) bajo una misma operación llamada **producto geométrico**.
- **Ventaja:** En lugar de inventar parches como el producto cruz o matrices específicas para cada dimensión, trata las reflexiones y rotaciones de forma natural y unificada en cualquier dimensión.

3. Tensores de orden superior

- **Qué son:** Una generalización multidimensional. Un escalar es un tensor de rango 0; un vector, de rango 1; una matriz, de rango 2. Los tensores de rango 3 o más son arreglos multidimensionales.
- **Ventaja:** Capturan relaciones multilineales simultáneas entre varios espacios vectoriales (esenciales en física de deformaciones, relatividad general y capas de redes neuronales).

4. Redes de grafos y operadores funcionales (Espacios de Hilbert)

- **Grafos de adyacencia / Operadores de difusión:** En lugar de multiplicar matrices densas, se interpretan los vectores como señales sobre nodos de una red y las operaciones como paso de mensajes locales.
- **Operadores diferenciales lineales:** Cuando los vectores tienen dimensión infinita (como funciones continuas), las "matrices" se convierten en operadores diferenciales o integrales (transformadas de Fourier, operadores de Laplace).

User prompt: aplicando esto al concepto de tokenizado y entendido de base el comportamiento fundamental de como los bytes componen las representacion que luego se convierten en datos, exploremos cual seria la formulacion adecuada o mas optimizada para el tokenizado de datos, ya no solo de lenguaje sino como abstracción general para el input de datos

Response: El tokenizado clásico (BPE, WordPiece, Unigram) es una heurística **discreta, miope y dependiente del dominio**: asume que segmentar secuencias de símbolos en un vocabulario finito de enteros es la mejor interfaz hacia el espacio vectorial continuo de un modelo.

Sin embargo, al concebir las matrices y tensores como transformadores continuos de representaciones, el tokenizado deja de ser un problema lingüístico y pasa a ser un **problema de compresión, proyección de señales y discretización óptima de flujos de bytes**.

1. El conflicto fundamental: Granularidad vs. Entropía

A nivel de hardware y almacenamiento, cualquier dato (texto, audio PCM, píxeles RGB, pesos, telemetría) es un flujo de bytes discretos: $B = (b_1, b_2, \dots, b_N)$ donde $b_i \in \{0, \dots, 255\}$.

- **A nivel de byte puro ($V = 256$):**
  - *Ventaja:* Vocabulario mínimo, cero tokens fuera de vocabulario (OOV), agnóstico al formato.
  - *Coste:* La longitud de secuencia N explota. En arquitecturas cuadráticas o incluso con atención lineal/estados recurrentes, la densidad de información por paso es bajísima, forzando a la red a gastar capas iniciales aprendiendo cómo ensamblar un entero de 32 bits o un carácter UTF-8.
- **A nivel de subpalabras/bloques estáticos ($V \approx 32\text{k} - 256\text{k}$):**
  - *Ventaja:* Comprime la secuencia temporal en un factor de $4\times$ a $8\times$.
  - *Coste:* Vocabulario hipertrofiado (matrices de embedding enormes), miopía composicional (errores ortográficos o prefijos hexadecimales rompen la coherencia) y rigidez absoluta frente a datos no previstos.

2. Formulación abstracta: El tokenizador como compresión con pérdida controlada

Un tokenizado general y óptimo para cualquier dato debe modelarse matemáticamente como una función en dos etapas: **segmentación adaptativa** seguida de **proyección a un espacio vectorial latente**.

Sea una secuencia continua o discreta de bytes $X = (x_1, \dots, x_N)$.

A. Segmentación por Entropía Local y Límites de Coherencia

En lugar de heurísticas como frecuencia de pares contiguos (BPE), la segmentación óptima para datos heterogéneos agrupa bytes evaluando el contenido de información (*surprisal*) condicional:

$$\tau_k = \arg\min_t \left(H(x_t \mid x_{<t}) - \lambda \cdot \text{len}(x_t) \right)$$

Los cortes entre "tokens" ocurren donde la predictibilidad colapsa (límites de un paquete binario, cambios de frecuencia en audio, bordes de un parche en una imagen). Donde la redundancia es altísima (repeticiones de ceros, secuencias de cabeceras predecibles), el bloque se expande dinámicamente sin gastar tokens de cómputo.

B. Operador de Proyección Vectorial (Byte-Embedding Continuo)

En vez de asignar cada segmento a un ID arbitrario de una tabla estática (lo cual desconecta $(254, 12)$ de $(254, 13)$), un bloque de bytes $S_k = [b_1, \dots, b_m]$ debe mapearse directamente al espacio latente continuo $\mathbb{R}^d$:

$$z_k = \mathbf{W}_{\text{proj}} \cdot \phi(S_k)$$

Donde ϕ puede ser:

- Un **kernel convolucional o pooling multiescala** sobre la secuencia de bytes.
- Una **transformación afín aprendida** sobre representaciones binarias/bit-plane (descomponiendo cada byte en sus 8 bits para preservar correlaciones de magnitud aritmética).

3. Técnicas avanzadas hacia la tokenización unificada

Si salimos de la tabla de enteros estática, las formulaciones más prometedoras para inputs universales se dividen en tres paradigmas:

Paradigma	Mecanismo Central	Ventaja como Abstracción Universal	Desafío Matemático/Ingeniería
Token-free / Byte-Latent (e.g., Megabyte, Mamba-Byte)	Subredes locales rápidas agregan K bytes antes de pasarlos a la red principal.	Elimina el concepto de token. Maneja binarios nativos sin preprocessing.	Enrutamiento dinámico cuando la entropía del dato no es uniforme.
Vector Quantization Adaptativa (FSQ / RVQ)	Cuantización residual de proyecciones continuas en códigos finitos.	Codifica cualquier señal continua (audio, imagen, texto continuo) en códigos de longitud fija.	Colapso de codebooks; requiere entrenar el cuantizador con una pérdida de reconstrucción.
Tokenizado por Context-Free Grammars / LZ Dinámico	Extracción algorítmica de invarianzas basadas en complejidad de Kolmogórov.	Comprime datos estructurados binarios, código y texto con garantías teóricas de compresión óptima.	No diferenciable de extremo a extremo; cómputo secuencial en CPU difícil de paralelizar.

4. La formulación matemática más eficiente: Tokenización Latente Diferenciable

Para abstraer el input sin sesgo de modalidad, la formulación teóricamente más sólida es **abandonar el tokenizado como paso previo desconectado** y convertirlo en la primera capa entrenable del modelo:

1. **Entrada cruda:** Secuencia de bytes $B \in \{0,1\}^{N \times 8}$ (representación explícita de bits).
2. **Proyección local por parche variable:** Un operador local f_θ (pequeño bloque recurrente o convolucional 1D) procesa ventanas de bytes y emite un escalar de "confianza de límite" $\alpha_i \in [0,1]$.
3. **Agrupamiento suave (*Soft-Pooling*):**

$$v_k = \sum_{j \in \text{segmento}_k} w_j \cdot \text{MLP}(b_j)$$

   donde la frontera no es un entero rígido, sino una mezcla ponderada que colapsa secuencias de baja entropía en un único vector en $\mathbb{R}^d$.
4. **Resultado:** El modelo central opera siempre sobre tensores en $\mathbb{R}^{M \times d}$ con $M \ll N$, donde cada posición latente representa un "trozo de información coherente", ya sea una palabra, un píxel codificado, un operando de 64 bits o una instrucción de ensamblador.

User prompt: la unidad es el bit y el convenio general es agruparlo en octetos que llamamos bytes pero 2 al 0 si? pero que pasa si flexibilizamos la composicion del byte de acuerdo al dominio? cada dominio tendria su tokenizado optimizado no?

Response: Exacto. El byte de 8 bits ($2^8 = 256$ estados) es un **convenio histórico de hardware** (diseñado originalmente por IBM en los años 60 para equilibrar texto alfanumérico ASCII, dígitos decimales empaquetados y direccionamiento de memoria alineado a potencias de 2), no una ley natural de la teoría de la información.

En términos de información estricta, la unidad base es el **bit** (un dígito binario con estados 2^0 o 2^1, con entropía máxima de 1 shannon). Si rompes la restricción arbitraria de empaquetar estrictamente en bloques de 8 bits y permites que el "ancho del símbolo" (k bits, con 2^k estados) se adapte a la física o estructura de cada dominio, obtienes la base teórica de un **tokenizado óptimo por dominio**.

¿Por qué forzar 8 bits destruye la eficiencia de la representación?

Cuando fuerzas cualquier dato a caber en bloques de 8 bits:

1. **O desperdicias capacidad (subutilización):** En ADN solo hay 4 bases (A, C, G, T). Cada base necesita exactamente $\log_2(4) = 2 \text{ bits}$. Guardar una base en un byte estándar de 8 bits desperdicia 6 bits de entropía por posición (75% de redundancia vacía) o fuerza a la red a descifrar representaciones ASCII redundantes (`0x41`, `0x43`, etc.).

2. **O fragmentas la entidad atómica (sobresegmentación):** Un valor de punto flotante de media precisión (`FP16`) o un sensor de 12 bits se corta a la fuerza en 2 bytes independientes. El modelo receptor tiene que gastar atención y parámetros aprendiendo a correlacionar el byte más significativo (`MSB`) con el byte menos significativo (`LSB`) para reconstruir la magnitud aritmética real.

Mapeo de dominios con su "byte" (símbolo) nativo óptimo

Si redefinimos el tamaño del bloque básico (k bits) según el alfabeto canónico de la fuente:

Dominio	Unidad atómica real	Tamaño natural (k bits)	Espacio de estados (2^k)	Desajuste frente al byte tradicional (8 bits)
Genómica / ADN	Nucleótido (A, C, G, T)	**2 bits**	4	El byte de 8 bits desperdicia el 75% del espacio si usa ASCII, o empaqueta 4 bases de forma rígida.
Audio PCM	Muestra de amplitud	**16 o 24 bits**	65.536 o $16.777.216$	Partir una muestra lineal de 24 bits en 3 bytes independientes destruye la continuidad numérica de la onda.
Imágenes médicas / Sensores RAW	Profundidad de píxel (ADC)	**10 o 12 bits**	1.024 o 4.096	No encaja en múltiplos de 8 bits sin relleno (*padding*) o desplazamiento de bits (*bit-packing* desalineado).
Juegos de tablero (Ajedrez / Go)	Estado de casilla / Tipo de pieza	**3 a 6 bits**	8 a 64	El byte normal transporta ceros irrelevantes en la mayoría de sus posiciones.
Instrucciones RISC	Opcode y registros fijos	**32 bits fijos**	2^{32}	Tratar binarios ARM/RISC-V a nivel de bytes ciegos oscurece la semántica de la instrucción completa.

La formulación del "Tokenizado Simbólico Óptimo"

Desde el punto de vista matemático, el tokenizado ideal de un dominio no es un vocabulario arbitrario, sino un **problema de cuantización y codificación de fuentes** (Teorema de Shannon):

1. Longitud fraccionaria de bits (Codificación Aritmética / ANS)

En la práctica, la información rara vez tiene una entropía que sea exactamente una potencia entera de 2 (2^k).

- Si un símbolo tiene una probabilidad $P(s)$, su contenido de información es:

$$I(s) = -\log_2 P(s) \text{ bits}$$

- Por ejemplo, si un evento ocurre el 70% de las veces, transporta ≈ 0.51 bits.
- Un sistema de empaquetado verdaderamente óptimo no asigna ni 8 bits ni 1 bit fijo: usa técnicas continuas de bits fraccionarios como **ANS (Asymmetric Numeral Systems)** o **codificación aritmética** para compactar el flujo continuo antes de proyectarlo al espacio latente.

2. Proyección directa sin tabla de embeddings (Bit-Plane Projection)

En lugar de crear una matriz de búsqueda estática (`nn.Embedding`) que requiere fijar cuántos tokens distintos existen:

- Tomas el bloque nativo de k bits del dominio expresado como un vector binario:

$$\mathbf{b} \in \{0, 1\}^k$$

- Lo proyectas directamente al espacio latente continuo $\mathbb{R}^d$ mediante una transformación afín:

$$\mathbf{z} = \mathbf{W}_{\text{proj}}\mathbf{b} + \mathbf{b}_{\text{bias}}$$

  donde $\mathbf{W}_{\text{proj}} \in \mathbb{R}^{d \times k}$.

Al hacer esto:

- Si $k = 2$ (ADN), $\mathbf{W}_{\text{proj}}$ solo tiene $d \times 2$ parámetros.
- Si $k = 16$ (audio), cada bit contribuye como una base ortogonal ponderada ($2^0, 2^1, \ldots, 2^{15}$), preservando de forma natural la topología métrica de los números en vez de tratar el valor `4001` y el `4002` como dos tokens categóricos completamente inconexos.

El trade-off real: ¿Por qué la industria sigue aferrada a los 8 bits?

Si el empaquetado por dominio es teóricamente superior, la razón de no usarlo en todos lados se reduce a la arquitectura de hardware:

1. **Alineación de memoria y SIMD:** Los procesadores (CPUs, GPUs, TPUs) están físicamente cableados para leer registros en múltiplos de 8, 16, 32 o 64 bits. Direccionar un flujo a nivel de 3, 5 o 12 bits requiere operaciones continuas de desplazamiento (`shift`) y máscaras lógicas (`AND`), lo que añade una penalización de rendimiento en la etapa de carga de datos (*data loader bottleneck*).
2. **Transferencia entre modalidades:** Un sistema multimodal que deba ingerir código ensamblador (32 bits), texto comprimido (variable), espectrogramas (16 bits) y secuencias lógicas necesitaría multiplexores de hardware dedicados en la capa de entrada.

A pesar de esa limitación de hardware, la tendencia moderna en arquitecturas que operan sin tokenizador convencional (*byte-level / token-free models*) va exactamente en esa dirección:

capas de entrada especializadas (*patchers* de bits o de muestras) que agrupan los bits según la física del sensor o dominio, convirtiéndolos en vectores densos antes de pasárselos al núcleo del modelo.

User prompt: debe haber alguna propiedad matematica de la quenos podamos valer para que la arquitectura cableada no sea la restricción

Response: Sí, la hay. La aparente restricción del hardware surge de un choque conceptual: **tratar el hardware como un manipulador de memoria alineada (punteros y desplazamientos de bits en CPU) en lugar de tratarlo como un motor de álgebra lineal continua (GPUs y aceleradores tensoriales)**.

El hardware moderno no está optimizado para desarmar bits secuencialmente; está optimizado para **multiplicar matrices a escala masiva**.

Para desacoplar el ancho del símbolo (k bits arbitrarios) de la rejilla rígida de la máquina ($2^n \in \{8, 16, 32, 64\}$ bits), nos valemos de varias propiedades matemáticas fundamentales:

1. Descomposición Polinomial y Homomorfismo de Empaquetado

Cualquier bloque de datos de la máquina (un registro SIMD de 32 o 64 bits) no es más que la evaluación de un polinomio en la base 2:

$$V = \sum_{i=0}^{W-1} b_i 2^i \quad \text{donde } b_i \in \{0, 1\}$$

Si tu dominio nativo tiene símbolos de longitud arbitraria k (por ejemplo, $k = 12$ para un sensor, o $k = 3$ para un estado discreto), el flujo continuo de bits no necesita "desempaquetarse" lógicamente mediante bucles de `shift` y `mask`.

La multiplicación matriz-vector es **lineal y distributiva**. Si proyectamos un registro alineado V de 32 bits que contiene símbolos de k bits empalmados, podemos absorber la reordenación, el desfase de bits y la escala aritmética en una única matriz de proyección fija $\mathbf{W}_{\text{proj}}$:

$$\mathbf{z} = \mathbf{W}_{\text{proj}} \cdot \mathbf{v}_{\text{raw}}$$

En lugar de reconstruir los símbolos en la CPU antes del cálculo, **la base de proyección efectúa la descomposición de bits implícitamente en el espacio continuo**. Los factores de escala 2^{-j} y los solapamientos entre límites de registros se convierten en coeficientes fijos precalculados en las columnas de la matriz.

2. Proyecciones Cuasi-Ortogonales (El lema de Johnson-Lindenstrauss)

Si el objetivo final del tokenizado no es recrear enteros sino alimentar un espacio de atención o recurrencia, **no necesitas reconstruir el símbolo exacto**.

Por el **Lema de Johnson-Lindenstrauss** y los principios de la computación hiperdimensional (*Vector Symbolic Architectures* / VSA):

- Un espacio de dimensión suficientemente alta d contiene un número exponencial de vectores que son casi ortogonales entre sí:

$$\langle \mathbf{u}_i, \mathbf{u}_j \rangle \approx 0 \quad (i \neq j)$$

- Puedes proyectar directamente palabras nativas de la máquina (por ejemplo, enteros de 32 o 64 bits leídos a máxima velocidad por SIMD/coalescencia de memoria) a través de una matriz de proyección aleatoria fija o convolución 1D fija:

$$\mathbf{z}_t = \mathbf{\Phi} \cdot \mathbf{x}_{\text{memoria}}$$

La topología de las distancias relativas entre patrones de bits se preserva en $\mathbb{R}^d$ **sin importar dónde caiga el límite del byte**. El modelo no necesita saber dónde terminó el símbolo k; la colisión de patrones se vuelve estadísticamente irrelevante debido a la concentración de la medida en altas dimensiones.

3. Dualidad Convolucional en el Dominio de Fourier (Teorema de Convolución)

La lectura desalineada de bits continuos sobre registros discretos es, en esencia, una operación de **correlación cruzada continua muestreada de forma discreta**.

Si representas el flujo continuo como una señal unidimensional, procesar patrones con periodicidad k a través de un canal rígido de período M (los 8, 32 o 64 bits del bus) genera un patrón de batido (interferencia de frecuencias).

Por el **Teorema de Convolución**:

$$\mathcal{F}\{f * g\} = \mathcal{F}\{f\} \cdot \mathcal{F}\{g\}$$

En lugar de alinear bits en el dominio temporal/espacial mediante operaciones lógicas secuenciales:

1. Se carga el flujo de datos directamente en bloques alineados nativos del hardware (sin tocar los bits).
2. Se aplica una Transformada Rápida de Fourier (o transformada equivalente sobre campos finitos / Walsh-Hadamard).
3. En el dominio frecuencial, cualquier desfase arbitrario de k bits se traduce simplemente en una **rotación de fase** (multiplicación por un escalar complejo $e^{-i\omega\Delta}$).

El alineamiento de bits deja de ser un problema secuencial de CPU (`O(N)` instrucciones con ramificaciones) y pasa a ser una multiplicación de fase en bloque totalmente paralelizable en GPU.

4. Sistemas Numéricos Asimétricos (ANS) y Dualidad de Autómatas

La formulación de **ANS (Asymmetric Numeral Systems)** demuestra matemáticamente que un flujo de símbolos con cualquier distribución de probabilidad o longitud fraccionaria de bits puede representarse como un único número entero de precisión arbitraria X.

El paso de añadir o extraer información se rige por una función reversible simple:

$$X' = C(s, X) \approx \frac{X}{P(s)}$$

Esta operación puede formularse no como manipulación de bits, sino como una **cadena de transformaciones lineales afines** sobre el estado:

$$\mathbf{s}_{t+1} = \mathbf{A}\mathbf{s}_t + \mathbf{B}\mathbf{u}_t$$

Esto transforma el problema del empaquetado y desempaquetado de secuencias no estándar en la misma formulación de un **modelo de espacio de estados (SSM)**.

Conclusión

La restricción del cableado solo existe si obligamos a la máquina a ejecutar la ruta clásica:

$$\text{Memoria} \xrightarrow{\text{shift/mask}} \text{Entero } k\text{-bits} \xrightarrow{\text{Lookup Table}} \text{Vector en } \mathbb{R}^d$$

Aprovechando la **linealidad de la proyección tensorial**, la **concentración de medida** y el **dominio de frecuencias**, el pipeline óptimo colapsa en un solo paso:

$$\text{Memoria en bruto} \xrightarrow{\mathbf{W}_{\text{invariante de fase}}} \text{Vector en } \mathbb{R}^d$$

La máquina lee a su velocidad nativa de bus (32/64/128 bits coalescentes), y las propiedades geométricas de la matriz se encargan de desacoplar, alinear e interpretar la física real del dato.

User prompt: para los casos en que k = a un numero no compatible con el cableado lo mejor no seria dsde el punto de vista de las matrices sintetizar de manera dinamica el proceso matematico mas optimo de tokenizado. No estoy buscando una respuesta, estoy buscando un proceso o metodologia

Response: Para formalizar este problema no como un parche algorítmico, sino como una **metodología de síntesis dinámica de operadores lineales**, el objetivo es derivar sistemáticamente una matriz de proyección $\mathbf{W}_{\text{opt}}(k)$ que mapee bloques leídos a velocidad de bus (W bits) directamente al espacio de representación latente ($\mathbb{R}^d$), absorbiendo la desalineación de k bits sin penalización de cómputo en CPU.

La metodología se estructura en cinco fases matemáticas sucesivas:

Fase 1: Caracterización Algebraica de la Incompatibilidad (El Grupo de Fase)

Sea W el ancho del registro nativo de hardware ($W \in \{32, 64, 128, 256\}$ bits) y k el ancho irreducible del símbolo del dominio, con $\gcd(k, W) < k$.

1. **Cálculo del ciclo canónico de fase:** El flujo continuo de bits crea un patrón periódico de desfase entre los límites del hardware y los límites del símbolo. El tamaño del superbloque irreducible en bits es el mínimo común múltiplo:

$$L_{\text{ciclo}} = \text{lcm}(k, W)$$

2. **Definición del espacio de estados de desfase:** El número de registros nativos involucrados es $M = L_{\text{ciclo}}/W$, y el número de símbolos emitidos es $S = L_{\text{ciclo}}/k$. Cada símbolo $s \in \{0, \dots, S-1\}$ posee un desplazamiento relativo δ_s:

$$\delta_s = (s \cdot k) \pmod{W}$$

Este conjunto de desfases define el grupo cíclico de transformaciones $\mathcal{G}_{k,W} \cong \mathbb{Z}_{\frac{W}{\gcd(k,W)}}$.

Fase 2: Formulación del Operador de Desempaquetado Implícito

En lugar de manipular bits, modelamos la extracción como una proyección tensorial.

1. **Representación del registro en el espacio euclidiano:** Un bloque contiguo de M palabras nativas se concatena en un vector de bits crudos $\mathbf{x} \in \{0,1\}^{L_{\text{ciclo}}}$.

2. **Construcción de la matriz base de muestreo $\mathbf{S}_k$:** Se define una matriz binaria o de coeficientes booleanos $\mathbf{S}_k \in \mathbb{R}^{S \cdot k \times L_{\text{ciclo}}}$ que formaliza la permutación de cableado:

$$\mathbf{y}_{\text{símbolos}} = \mathbf{S}_k \mathbf{x}$$

Dado que $L_{\text{ciclo}} = S \cdot k$, $\mathbf{S}_k$ es una matriz de permutación canónica o cuasi-identidad con desfases periódicos dictados por δ_s.

Fase 3: Plegado Matricial (*Operator Folding*) y Reducción Dimensional

Esta es la fase crítica donde el desempaquetado y el embedding colapsan en una sola transformación.

1. **Fusión con la base del dominio:** Si cada símbolo k se evalúa mediante una matriz de embedding o proyección al espacio latente $\mathbf{E} \in \mathbb{R}^{d \times k}$, la proyección global sin optimizar sería:

$$\mathbf{Z} = \mathbf{E} \cdot (\mathbf{S}_k \mathbf{x})$$

2. **Síntesis del operador colapsado $\mathbf{\Omega}$:** Por la asociatividad del producto matricial, precalculamos dinámicamente:

$$\mathbf{\Omega}(k) = (\mathbf{I}_S \otimes \mathbf{E}) \cdot \mathbf{S}_k \quad \in \mathbb{R}^{(S \cdot d) \times L_{\text{ciclo}}}$$

Donde $\otimes$ denota el producto de Kronecker.

3. **Descomposición en valores singulares (SVD) / Factorización de bajo rango:** Para eliminar la redundancia estructural de los ceros y alineamientos periódicos en $\mathbf{\Omega}(k)$:

$$\mathbf{\Omega}(k) \approx \mathbf{U}_r \mathbf{\Sigma}_r \mathbf{V}_r^T$$

Esto factoriza la lectura de memoria desalineada en un producto de dos matrices densas y compactas de rango $r \ll L_{\text{ciclo}}$, optimizadas para los núcleos tensoriales (Tensor Cores / Systolic Arrays).

Fase 4: Compensación Espectral y Corrección de Aliasing

Dado que muestrear símbolos de k bits a través de una rejilla de W bits induce armónicos espaciales (el equivalente discreto al batido en procesamiento de señales):

1. **Transformación al dominio cuasi-ortogonal (Walsh-Hadamard o DFT 1D):** Se aplica una base ortogonal rápida $\mathbf{H}_W$ sobre los registros nativos cargados.
2. **Alineación por modulación de fase:** En el dominio transformado, cualquier desfase δ_s se convierte en una multiplicación punto a punto por un vector de coeficientes de rotación $\mathbf{c}_{\delta_s}$:

   $$\hat{\mathbf{z}}_s = \mathbf{c}_{\delta_s} \odot (\mathbf{H}_W \mathbf{x}_{\text{reg}})$$

   Esto elimina la necesidad de reorganizar bits en memoria física: la corrección geométrica del desfase ocurre como una rotación en el espacio espectral.

Fase 5: Compilación Dinámica del Kernel Tensorial (Pipeline en Tiempo de Ejecución)

La metodología concluye transformando la parametrización matemática en ejecución de hardware:

1. **Entrada de metadatos:** El sistema recibe el par (k, W) en tiempo de compilación o inicialización de inferencia.
2. **Síntesis analítica:** Se calcula la matriz dispersa compacta $\mathbf{\Omega}(k)$ y sus coeficientes de rotación espectral.
3. **Instanciación del Kernel:**
   - La GPU/acelerador ejecuta una lectura coalescente estándar de bloques W.
   - La primera capa densa del modelo aplica directamente la multiplicación $\mathbf{Z} = \mathbf{\Omega}(k)\mathbf{X}_{\text{bus}}$, proyectando los datos en bruto directamente a los S vectores latentes de dimensión d.
4. **Resultado:** El costo computacional del "desempaquetado" no estándar queda absorbido dentro de la primera proyección lineal estándar de la red, logrando una complejidad marginal de preprocesamiento idéntica a cero ($O(1)$ transferencias de memoria).
